

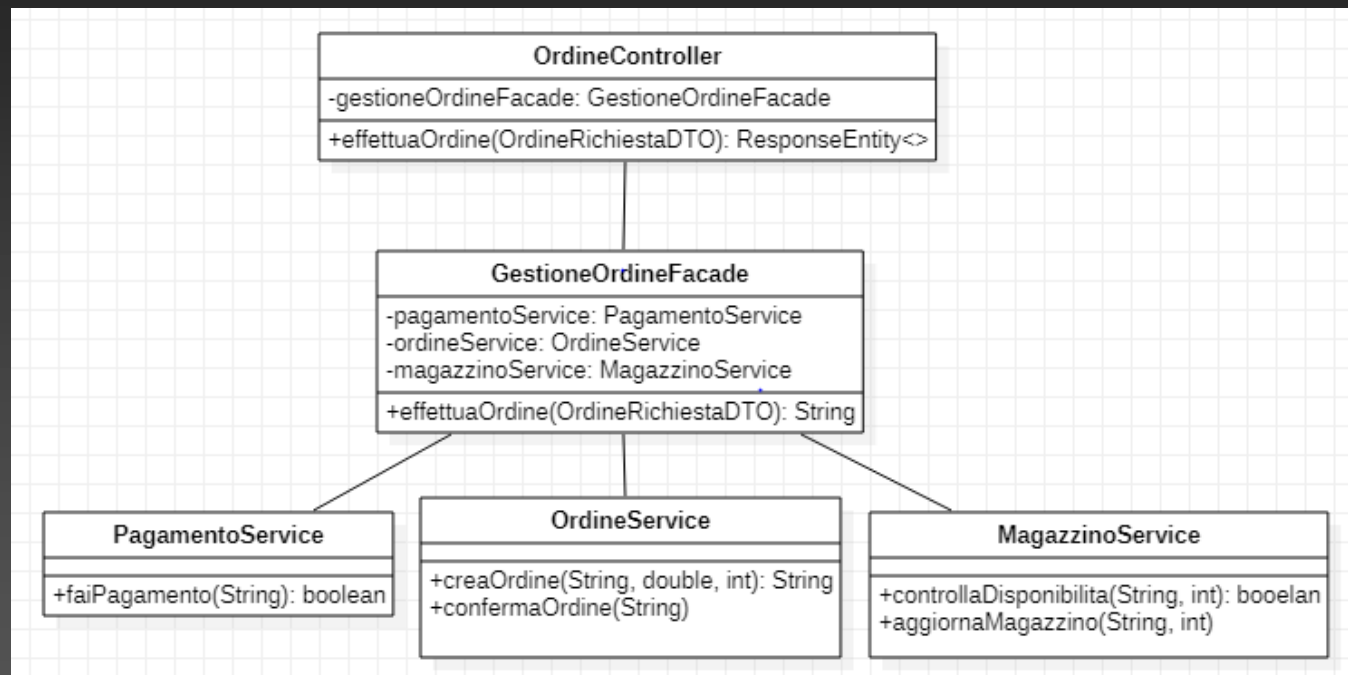
Spring Boot

Spring Web RESTful

Facade Pattern

- Facade in inglese significa «facciata».
- E' un design pattern strutturale.
- Usiamo un facade quando dobbiamo implementare un servizio complesso che internamente fa uso di altri servizi.
- Vantaggi:
 - Nasconde al componente che fa uso della facade la complessità che essa contiene.
 - Disaccoppia il componente che fa uso della facade dal servizio complesso. Le modifiche lasciano intatto il componente client.
 - Per usare un servizio interno della facade non dobbiamo per forza passare su di essa.

Facade Pattern



Facade Pattern



```
@Service
public class PagamentoServiceImpl implements PagamentoService {
    @Override
    public boolean faiPagamento(String datiCarta) {
        // Business logic per effettuare il
        return true;
    }
}
```

```
@Service
public class MagazzinoServiceImpl implements MagazzinoService {
    @Override
    public boolean controllaDisponibilita(String prodotto, int quantita) {
        // Business logic per il controllo della quantità in magazzino
        return true;
    }

    @Override
    public void aggiornaMagazzino(String prodotto, int quantita) {
        // Business logic per l'aggiornamento della quantità in magazzino
    }
}
```

```
@Service
public class OrdineServiceImpl implements OrdineService{
    @Override
    public String creaOrdine(String prodotto, double prezzo, int quantita){
        // Business logic per dell 'ordine
        return "numero ordine xxx123" ;
    }

    @Override
    public void confermaOrdine(String numeroOrdine) {
        // Business logic per la conferma dell 'ordine
    }
}
```



Facade Pattern

```
@Service
public class GestioneOrdineFacadeImpl implements GestioneOrdineFacade {
    @Autowired
    public MagazzinoService magazzinoService;
    @Autowired
    public PagamentoService pagamentoService;
    @Autowired
    public OrdineService ordineService;

    @Transactional
    @Override
    public String effettuaOrdine(OrdineRichiestaDTO ordineDto) {
        boolean disponibile = magazzinoService.controllaDisponibilita(ordineDto.getProdotto(), ordineDto.getQuantita());
        if (!disponibile) throw new RuntimeException(HttpStatus.NOT_ACCEPTABLE, "prodotto non disponibile");
        String numeroOrdine = ordineService.creaOrdine(ordineDto.getProdotto(), ordineDto.getPrezzo(), ordineDto.getQuantita());
        pagamentoService.faiPagamento(ordineDto.getDatiCarta());
        ordineService.confermaOrdine(numeroOrdine);
        magazzinoService.aggiornaMagazzino(ordineDto.getProdotto(), ordineDto.getQuantita());
        return numeroOrdine;
    }
}
```

Facade Pattern

```
@RestController
@RequestMapping( "/api/ordine" )
public class OrdineController {
    @Autowired
    public GestioneOrdineFacade facade;

    @PostMapping("/crea")
    public ResponseEntity<Map<String, Object>> effettuaOrdine(@RequestBody OrdineRichiestaDTO ordineDto) {
        String numeroOrdine = facade.effettuaOrdine(ordineDto);
        Map<String, Object> response = new HashMap<>();
        response.put("timestamp", LocalDateTime.now());
        response.put("status", HttpStatus.CREATED.value());
        response.put("numero_ordine", numeroOrdine);
        return new ResponseEntity<>(response, HttpStatus.CREATED);
    }
}
```

Facade Pattern



HTTP request

```
{
  body
  .... "prodotto": "prodotto_xxx",
  .... "quantita": "2",
  .... "prezzo": "20.5",
  .... "datiCarta": "1234567891015347"
}
```

HTTP response body

```
{
  "numero_ordine": "numero_ordine_xxx123",
  "timestamp": "2022-10-05T08:48:08.165+00:00",
  "status": 201
}
```

